Churn Analysis and Customer Intelligence

A Data Analytics Project Report

Data Analytics Team

Tools: Python · SQLite · Pandas · Matplotlib · Seaborn

August 2026

Project Summary

Project Type:	End-to-End Customer Churn Analysis
Database:	SQLite (`customer_churn.db`)
Dataset Size:	21 customers × 3 relational tables
Primary Language:	Python 3 (Jupyter Notebook)
Geography:	India (multiple states)
Churn Rate:	28.6% (6 of 21 customers churned)

Contents

1 Introduction

1.1 Project Background

Customer churn — the phenomenon by which customers discontinue their relationship with a service provider — is one of the most critical metrics in subscription-based businesses. Acquiring new customers typically costs five to seven times more than retaining existing ones, making churn analysis an indispensable component of any data-driven customer success strategy.

This project undertakes a comprehensive, end-to-end churn analysis for a fictional subscription service, using a relational SQLite database containing customer demographics, subscription details, and support interaction records. The analysis spans the full data analytics pipeline: from raw data ingestion and cleaning, through feature engineering and exploratory analysis, to actionable business insights.

1.2 Objectives

The principal objectives of this project are:

1. **Data Integration**: Consolidate customer, subscription, and support data from a relational SQLite database into a unified analytical dataset.
2. **Data Quality**: Identify and remediate data quality issues including missing values, type inconsistencies, duplicate records, and non-standard categorical encodings.
3. **Feature Engineering**: Derive analytically meaningful features from raw fields to enrich the modelling and exploratory analysis.
4. **Exploratory Data Analysis (EDA)**: Uncover distributional patterns, segment-level churn rates, financial metrics, and cross-variable correlations.
5. **Visualization**: Communicate findings through well-designed charts covering temporal, geographic, and segment-based perspectives.
6. **Business Insights**: Translate statistical findings into concrete, prioritized recommendations for reducing churn.

1.3 Scope

The scope of the project is confined to descriptive and diagnostic analytics. Predictive modelling (e.g., machine learning classifiers) is identified as a recommended next step but falls outside the current deliverable. All 21 customer records are from India, restricting geographic generalizations to the Indian market context.

2 Data Source and Description

2.1 Database Overview

The dataset is stored in a SQLite relational database file, `customer_churn.db`, comprising three interrelated tables. All three tables share the `customerid` field as the join key, enabling a full relational merge into a single analytical DataFrame of **21 rows × 21 columns**.

2.2 Table Schemas

Table 1: Database Table Overview

Table Name	Domain	Rows	Key Columns
`db_customer`	Demographics	21	`customerid, name, country, state, gender, dob, interests, pincode`
`db_subscription`	Subscription	21	`customerid, subscription_start_date, plan_type, contract_type, cancellation_date, monthly_charges, cltv, churn_score`
`db_support`	Customer Support	9	`customerid, complaint_date, escalations, csat_score, comment`

2.3 Table Descriptions

db_customer. This table captures static demographic attributes for all 21 customers. Notable fields include `gender`, `state` (Indian state of residence), and `dob` (date of birth). The `interests` and `pincode` fields were found to contain a high rate of null values and were subsequently dropped during preprocessing.

db_subscription. This is the central analytical table, containing each customer's subscription lifecycle data. The `plan_type` (Basic, Standard, Premium) and `contract_type` (Monthly, Annual) fields define the service tier. `cancellation_date` serves as the primary binary churn indicator, while `churn_score` provides a pre-computed numeric risk score. `monthly_charges` and `cltv` (Customer Lifetime Value) provide the financial dimension.

db_support. This table records customer support interactions for 9 of the 21 customers (some customers had no complaints on record). Each record includes a `complaint_date`, a binary `escalations` indicator (Y/N), a `csat_score` (Customer Satisfaction score), and a free-text `comment`. Multiple complaint records per customer were de-duplicated during preprocessing.

2.4 Target Variables

The analysis uses two complementary churn targets:

- `churn_score`: A continuous numeric score representing churn risk (pre-computed, source methodology unspecified).
- `churn_flag`: A binary feature (0 = active, 1 = churned) engineered from `cancellation_date` — the primary target for segment-level analysis.

3 Methodology

3.1 Technology Stack

The analysis was implemented entirely in Python 3 within a Jupyter Notebook environment, leveraging the following libraries:

Table 2: Technology Stack and Roles

Library / Tool	Version	Role in Project
Python 3	3.x	Core programming language
Jupyter Notebook	–	Interactive development environment
`sqlite3`	stdlib	SQLite database connectivity
`pandas`	–	Data manipulation and analysis
`numpy`	–	Numerical operations
`matplotlib`	–	Base charting and visualization
`seaborn`	–	Statistical visualization
`datetime`	stdlib	Date parsing and arithmetic

3.2 Analytical Workflow

The project followed a structured, sequential analytics pipeline as illustrated below:

Step 1. Data Ingestion — Connect to `customer_churn.db` via `sqlite3`, enumerate tables using `sqlite_master`, and load each table into a Pandas DataFrame using `pd.read_sql()`.

Step 2. Data Exploration — Inspect shapes, data types, null counts, and value distributions across all three tables.

Step 3. Data Preprocessing — Apply six targeted cleaning transformations (Section 4).

Step 4. Table Merging — Perform sequential left joins on `customerid` to produce a single master DataFrame.

Step 5. Feature Engineering — Derive six analytically relevant features from existing columns (Section 5).

Step 6. Exploratory Data Analysis — Compute summary statistics, segment-level aggregations, and correlation matrices (Section 6).

Step 7. Visualization — Produce five charts spanning temporal, segment, geographic, and correlation analyses (Section 7).

Step 8. Insight Generation — Synthesize findings into prioritized business recommendations (Section 9).

4 Data Preprocessing

Six preprocessing steps were applied sequentially to ensure data quality and analytical readiness.

4.1 Step 1 — Column Renaming

The `name` column in `db_customer` was renamed to `customer_name` to avoid conflicts with Python built-in functions and to improve code readability. This is a best-practice convention in data engineering.

4.2 Step 2 — Dropping Irrelevant Columns

Two columns were identified as unsuitable for analysis and removed:

- `interests`: High null rate; insufficient data density to derive meaningful signal.
- `pincode`: Granular geographic identifier with high cardinality and high null rate; not actionable at the available sample size.

Removing these columns reduces noise and prevents spurious correlations in downstream analysis.

4.3 Step 3 — Data Type Conversion

All date-related columns were converted from `object` (string) dtype to `datetime64[ns]` using `pd.to_datetime()`. Affected columns include:

- `dob` (date of birth)
- `subscription_start_date`
- `cancellation_date`
- `complaint_date`

Proper datetime typing is a prerequisite for all temporal feature engineering and date arithmetic performed in later steps.

4.4 Step 4 — Data Standardization (Gender Normalization)

Inconsistent categorical labels were found in the `gender` column. Non-standard entries were mapped to canonical values:

Original Value	Standardized Value
`"Men"`	`"Male"`
`"Women"`	`"Female"`

4.5 Step 5 — Missing Value Imputation (Country Field)

Three records in `db_customer` had null values in the `country` column. Since all customers are from India, these were imputed using a state-to-country lookup: for any record with a valid `state` value mapping to an Indian state, `country` was filled with `"India"`. This deterministic imputation approach is appropriate given the geographic scope of the dataset.

4.6 Step 6 — Duplicate Record Handling (Support Table)

The `db_support` table contained multiple complaint records for some customers. To prevent row duplication upon merging, duplicate records were resolved by retaining only the **latest complaint record per customer** (sorted by `complaint_date`, keeping `last`). This ensures a one-to-one mapping between customers and support records during the join operation.

5 Feature Engineering

Six new features were derived from the raw data to enrich the analytical dataset. These features transform implicit information into explicit, analytically tractable signals.

Table 3: Engineered Features Summary

Feature	Method	Description
`churn_flag`	Binary from `cancellation_date`	1 if `cancellation_date` is no (customer has churned); 0 othe Primary binary churn target.
`churn_risk`	Binning of `churn_score`	Categorical risk tier: *low*, *m high*, derived from numeric score thresholds.
`complaint_count`	Groupby aggregation per `customerid`	Total number of support ticket per customer (range: 0–2). tures support burden.
`tenure_days`	Date difference calculation	Days elapsed `subscription_start_date` ative to analysis date). Ca customer longevity.
`cancellation_month`	Period extraction from `cancellation_date`	Calendar month–year of cancel (as a period object). Enables t ral trend analysis.
`escalations`	Encoding: Y/N $\rightarrow$ 1/0	Binary flag indicating whet complaint was escalated. C churn predictor (correlation =

5.1 Feature Rationale

churn_flag. While `churn_score` provides a continuous risk estimate, a binary flag derived from `cancellation_date` provides a ground-truth outcome label. This allows for proportion-based analysis (churn rates) and direct segment comparison.

churn_risk. Segmenting the continuous `churn_score` into ordinal risk tiers (low/med/high) enables intuitive business reporting and facilitates ordinal encoding for correlation analysis.

complaint_count. Aggregated before the de-duplication step to capture total complaint volume per customer — an important signal for customer dissatisfaction that is lost if only the latest record is retained.

tenure_days. Customer tenure is a well-established churn predictor: newer customers tend to churn more readily than long-standing ones. This feature enables tenure-based segmentation.

escalations. The binary encoding of the Y/N escalations flag converts a string categorical into a numeric variable, making it compatible with correlation matrices and quantitative analysis. Its 0.77 correlation with churn makes it the strongest engineered predictor in this dataset.

6 Exploratory Data Analysis

6.1 Churn Overview

Table 4: Overall Churn Distribution

Status	Count	Percentage
Active (not churned)	15	71.4%
Churned	6	28.6%
Total	**21**	**100%**

Of the 21 customers in the dataset, **6 (28.6%) have churned** as indicated by a non-null `cancellation_date`. This is a notably elevated churn rate by industry standards, suggesting meaningful room for retention improvement.

6.2 Churn Rate by Plan Type

Table 5: Churn Rate by Subscription Plan

Plan Type	Total Customers	Churned	Churn Rate
Basic	5	3	**60.0%**
Standard	9	2	22.2%
Premium	7	1	**14.3%**
Overall	**21**	**6**	**28.6%**

The plan-type breakdown reveals a dramatic gradient: **Basic plan customers churn at 60%**, compared to just 14.3% for Premium subscribers — a **4.2× difference**. Standard plan customers sit in the middle at 22.2%. This pattern strongly suggests that lower-tier customers experience greater dissatisfaction or perceive lower value relative to their expectations.

6.3 Financial Metrics by Plan Type

Table 6: Total Monthly Charges by Plan Type

Plan Type	Customers	Total Monthly Charges (USD)
Basic	5	$52.95
Standard	9	$123.91
Premium	7	$218.93
Total	**21**	**$395.79**

Premium subscribers collectively contribute $218.93 in monthly recurring revenue — more than four times the Basic tier's contribution. Combined with their lower churn rate, Premium customers represent disproportionately high long-term value.

6.4 Customer Support & Satisfaction Analysis

Table 7: Support Interaction Summary

Metric	Value
Total support records	9
Unique customers with complaints	7
Average complaints per customer	0.43
Customers with escalations	Subset of 7
Escalation–Churn Correlation	**0.77**

The most striking finding from the support data is the **strong positive correlation (0.77) between escalations and churn**. This is among the highest feature correlations in the dataset and represents a highly actionable business signal: customers who escalate their complaints are far more likely to cancel their subscriptions.

6.5 Correlation Analysis

To enable a numeric correlation matrix, categorical variables were encoded using an ordinal (priority-weighted) scheme:

Table 8: Ordinal Encoding Scheme for Categorical Variables

Variable	Categories	Encoding
`plan_type`	Basic, Standard, Premium	0, 1, 2
`contract_type`	Monthly, Annual	0, 1
`churn_risk`	low, med, high	0, 1, 2
`escalations`	N, Y	0, 1

Two versions of the heatmap were produced: one with unweighted dummy encoding and one with the priority-weighted ordinal encoding above. The ordinal encoding better captures the natural ordering in plan tiers and risk levels, and is the preferred version for interpretation.

7 Data Visualization

Five visualizations were produced using `matplotlib` and `seaborn` to communicate key findings across temporal, segment-based, geographic, and multivariate dimensions.

7.1 Chart 1 — Monthly Churn Trend (Line Plot)

Purpose: To identify temporal patterns in when customers are cancelling their subscriptions.

Chart type: Line plot with markers, x-axis representing calendar months, y-axis representing the count of churned customers.

Key insights: The monthly trend chart reveals whether churn is uniformly distributed or concentrated in specific periods. Spikes in particular months may correspond to contract renewal windows, seasonal dissatisfaction, or service disruptions. This temporal view is essential for planning proactive retention campaigns ahead of high-risk periods.

7.2 Chart 2 — Churn Rate by Plan Type (Bar Chart)

Purpose: To compare churn rates across Basic, Standard, and Premium plan tiers.

Chart type: Horizontal or vertical grouped bar chart with churn rate (%) on the y-axis and plan type on the x-axis.

Key insights: This chart directly visualizes the 60% vs. 22.2% vs. 14.3% gradient across plan types. The visual contrast makes immediately clear that Basic plan subscribers are at substantially higher risk. This chart is the single most actionable visualization for product and pricing teams.

7.3 Chart 3 — Churn Rate by State (Bar Chart)

Purpose: To map the geographic distribution of churn across Indian states represented in the dataset.

Chart type: Bar chart with Indian state names on the x-axis and churn rate on the y-axis.

Key insights: Geographic variation in churn rates may reflect differences in market maturity, regional competition, connectivity quality, or support service levels. States with elevated churn rates warrant targeted regional campaigns or localized support improvements. This view is particularly relevant for a business operating across India's diverse regional markets.

7.4 Chart 4 — Correlation Heatmap (Unweighted Encoding)

Purpose: To visualize pairwise Pearson correlations across all numeric and dummy-encoded features.

Chart type: Seaborn heatmap with a diverging color scale (e.g., blue-white-red), annotated with correlation coefficients.

Key insights: The unweighted heatmap provides a broad view of feature relationships, including correlations between demographic variables, subscription features, and churn indicators. It serves as an exploratory compass for identifying which variable pairs are worth investigating further.

7.5 Chart 5 — Correlation Heatmap (Priority-Weighted Encoding)

Purpose: A refined correlation matrix using ordinal encoding to better reflect the natural ordering of plan tiers, contract types, and risk levels.

Chart type: Seaborn heatmap, identical structure to Chart 4 but with ordinal-encoded categorical variables.

Key insights: With ordinal encoding, the correlation between `plan_type` and `churn_flag` becomes more meaningful: higher plan tiers correlate negatively with churn. The escalation–churn correlation of 0.77 stands out prominently in this matrix as the strongest pairwise relationship. The priority-weighted heatmap is the recommended reference for feature selection in any future predictive modelling work.

8 SQL Operations in Python

8.1 Database Connectivity

Data was accessed using Python's built-in `sqlite3` module in conjunction with `pandas`. The connection and query pattern followed is a standard, production-grade approach:

1. Establish a database connection: `conn = sqlite3.connect("customer_churn.db")`
2. Enumerate available tables: `SELECT name FROM sqlite_master WHERE type='table'`
3. Load each table into a DataFrame: `pd.read_sql("SELECT * FROM db_customer", conn)`
4. Close the connection upon completion: `conn.close()`

8.2 Schema Exploration via `sqlite_master`

The `sqlite_master` system table was queried to programmatically list all tables in the database. This approach is preferred over hard-coding table names, as it is self-documenting and generalizes to databases with unknown schemas.

8.3 SQL-in-Python Demonstration

To demonstrate the full range of SQL-in-Python capabilities, an additional exercise was performed:

- **CREATE TABLE**: A test `users` table was created in memory using SQLite.
- **INSERT INTO**: Sample records were inserted into the `users` table.
- **SELECT**: The inserted records were queried back and loaded into a DataFrame using `pd.read_sql()`.

This exercise demonstrates the bidirectional data flow capability between Python data structures and SQL databases — a pattern applicable to writing analysis results back to a production database.

8.4 Advantages of SQLite for Analytics Projects

- **Zero configuration**: No server setup required; database is a single file.
- **Full SQL support**: Supports joins, aggregations, subqueries, and window functions.
- **Pandas integration**: Seamless interoperability via `pd.read_sql()` and `df.to_sql()`.
- **Portability**: The `.db` file can be shared, versioned, and deployed across platforms.

9 Key Findings and Business Insights

9.1 Summary of Key Findings

The following findings emerge from the exploratory data analysis:

F1. Overall churn rate is 28.6%. Six of 21 customers have cancelled their subscriptions, which is above typical industry benchmarks and signals a need for active retention measures.

F2. Basic plan customers churn at 4× the rate of Premium customers. The 60% churn rate on the Basic tier versus 14.3% on Premium is the most significant segment-level finding. It suggests that Basic plan customers may be under-served, dissatisfied with value, or unable to afford long-term commitment.

F3. Escalations are a leading indicator of churn (correlation = 0.77). This is the strongest pairwise relationship in the dataset. Customers who escalate support complaints are substantially more likely to cancel. Escalation events are observable in near-real-time, making this a highly actionable early warning signal.

F4. Premium customers contribute disproportionately to revenue. Total monthly charges for Premium subscribers ($218.93) are more than four times those of Basic subscribers ($52.95), reinforcing the business case for upselling and Premium retention.

F5. Churn varies meaningfully by Indian state. Geographic variation suggests that market-specific factors (regional competition, service quality, economic conditions) contribute to churn differentially across states.

F6. Average complaint volume is low (0.43 per customer), but the subset of customers who do complain—and especially those who escalate—have a dramatically elevated churn rate.

9.2 Business Recommendations

Based on the above findings, the following prioritized recommendations are offered:

R1. Intervene immediately on escalated support tickets. With a 0.77 escalation–churn correlation, each escalated complaint should trigger an automated retention workflow: a personal outreach call, a service credit offer, or a plan upgrade offer. The cost of this intervention is almost certainly lower than the cost of churn and re-acquisition.

R2. Redesign the Basic plan to reduce perceived value gap. A 60% churn rate on the Basic tier is unsustainable. Consider enriching the Basic plan's feature set, adjusting pricing, or introducing a transitional *Starter Plus* tier. Alternatively, use CLTV data to identify high-value Basic customers for proactive upgrade offers.

R3. Upsell Basic customers to Standard or Premium. Given that Standard (22.2%) and Premium (14.3%) customers churn far less, a structured upsell program targeting Basic subscribers—particularly those with high CLTV scores—could materially reduce overall churn while increasing revenue.

R4. Launch targeted retention campaigns in high-churn states. Geographic churn variation warrants state-level segmentation of retention campaigns. High-churn states should receive additional customer success resources, localized offers, or regional support improvements.

R5. Leverage CLTV for prioritized retention spend. The `cltv` field in the subscription table provides a ready-made customer value ranking. Retention resources should be disproportionately allocated to high-CLTV customers who are showing churn risk signals, maximizing return on retention investment.

R6. Monitor complaint_count as an early churn signal. Even before escalation, multiple complaints from a single customer may predict churn. Setting a threshold (e.g., ≥ 2 complaints) to trigger proactive outreach could catch at-risk customers before they escalate.

10 Conclusion

10.1 Summary

This project delivered a complete, end-to-end customer churn analysis using a SQLite-backed relational dataset of 21 Indian subscription customers. Starting from raw, multi-table data with quality issues, the analysis progressed through systematic preprocessing, feature engineering, and exploratory data analysis to produce actionable business insights.

The key finding — that **escalated support complaints correlate with churn at 0.77** and that **Basic plan subscribers churn at 60%** — provides two clear, measurable levers for a retention strategy: support intervention and plan optimization. These are not merely statistical observations; they are early warning signals observable in operational data that can drive real-time business decisions.

10.2 Limitations

- **Small sample size**: With only 21 customers, all findings are directional rather than statistically conclusive. Confidence intervals would be wide, and findings should be validated on a larger population before operationalizing.
- **No predictive modelling**: The current analysis is descriptive and diagnostic. It identifies *who* churns and *what correlates with churn*, but does not yet quantify *the probability* that a specific current customer will churn.
- **Single time period**: The dataset does not appear to span a long observation window, limiting seasonal and longitudinal analysis.

10.3 Future Directions

The following extensions would substantially increase the value of this analytical programme:

1. **Machine Learning Churn Prediction**: Train binary classifiers (Logistic Regression, Random Forest, XGBoost) on the engineered feature set to generate real-time per-customer churn probabilities. This would enable proactive, data-driven outreach rather than reactive intervention.
2. **Survival Analysis**: Apply Cox Proportional Hazards models or Kaplan–Meier estimators to model the *time to churn* distribution, enabling cohort-level survival curves by plan type and contract type.
3. **NLP on Support Comments**: Apply sentiment analysis or topic modelling to the free-text `comment` field in `db_support` to identify the categories of complaint most strongly associated with churn.
4. **A/B Testing Framework**: Design controlled experiments to test the effectiveness of specific retention interventions (e.g., upgrade offers, proactive outreach) identified through this analysis.

5. **Scale to Production**: Migrate from SQLite to a production database (PostgreSQL, BigQuery), implement automated feature refresh pipelines, and deploy churn scores via a REST API for integration with CRM systems.

This analysis was conducted using open-source Python tools.
All findings are based on the provided `customer_churn.db` *dataset.*

Appendix: Tools and Libraries Reference

Table 9: Python Libraries Used in This Project

Library	Purpose & Usage in This Project
`sqlite3`	Built-in Python module for SQLite database connectivity. Used to connect to `customer_churn.db`, query `sqlite_master`, and execute DDL/DML statements.
`pandas`	Core data manipulation library. Used for DataFrame creation, merging, groupby aggregations, filtering, and feature engineering.
`numpy`	Numerical computing. Used for array operations and mathematical computations underlying pandas operations.
`matplotlib`	Base visualization library. Used to configure figure sizes, axes, titles, and labels for all charts.
`seaborn`	Statistical visualization built on matplotlib. Used to produce correlation heatmaps and styled bar charts.
`datetime`	Built-in Python module. Used for date parsing, arithmetic (e.g., `tenure_days`), and period extraction (`cancellation_month`).
`Jupyter Notebook`	Interactive development and documentation environment. Provided inline visualization and iterative code execution.

Table 10: Database and Infrastructure

Component	Technology	Notes
Database Engine	SQLite 3	File-based, serverless RDBMS
Database File	`customer_churn.db`	Contains 3 tables, 21 customers
Development Environment	Jupyter Notebook	Interactive Python kernel
Programming Language	Python 3	Primary analysis language